Objetos en JavaScript

Un objeto es una estructura que contiene tanto las variables (llamadas propiedades) como las funciones que manipulan dichas variables (llamadas métodos). A partir de esta estructura se ha creado un nuevo modelo de programación (la programación orientada a objetos) que atribuye a las mismas propiedades como herencia o polimorfismo. Como veremos, JavaScript simplifica en algo este modelo y hace una programación híbrida entre la programación estructurada y la programación orientada a objetos.

El modelo de la programación orientada a objetos normal y corriente separa los mismos en dos: clases e instancias (objetos). Las primeras son entes más abstractos que definen un conjunto determinado de objetos. Las segundas son miembros de una clase, poseyendo las mismas propiedades que la clase a la cual pertenecen.

Propiedades y métodos.

Para acceder a los métodos y propiedades de un objeto debemos utilizar la siguiente sintaxis:

objeto.propiedad
objeto.metodo(parametros)

Conceptos Básicos.

Objetos: Son todas las cosas con identidad propia. Se relacionan entre si. Poseen características (atributos) y tienen responsabilidades (funciones, métodos) que deben cumplir.
Son ejemplares (instancias) de una clase y conocen a la clase a la cual pertenecen.

Atributos o propiedades: Son las características, cualidades distintivas de cada objeto. Deben ser mínimos para poder realizar todas las operaciones que requiere la aplicación.

Ejemplos de objetos del mundo real:

- Casa:
atributos: tamaño, precio, cantidad de habitaciones, etc.;
responsabilidades: comodidad, seguridad, etc.
- Mesa:
atributos: altura, largo, ancho, etc.;
responsabilidades: contener elementos.
- Ventana:
atributos: tamaño, color, etc.;

IES MARIA ENRIQUEZ

responsabilidades: abrirse, cerrarse, etc.

Ejemplos de objetos del mundo de la programación:

- Ventana:
atributos: tamaño, color, etc.;
responsabilidades: mostrar título,achicarse
etc.

Responsabilidades o Métodos.

Son las responsabilidades que debe cumplir la clase.

El objetivo de un método es ejecutar las actividades que tiene encomendada la clase.

Es un algoritmo (conjunto de operaciones) que se ejecuta en respuesta a un mensaje; respuestas a mensajes para satisfacer peticiones.
Un método consiste en el nombre de la operación y sus argumentos. El nombre del método identifica una operación que se ejecuta.
Un método está determinado por la clase del objeto receptor, todos los objetos de una clase usan el mismo método en respuesta a mensajes similares.

La interpretación de un mensaje (selección del método ejecutado) depende del receptor y puede variar con distintos receptores, es decir, puede variar de una clase a otra.

Clases

Una clase es un molde para objetos que poseen las mismas características (que pueden recibir los mismos mensajes y responden de la misma manera).

Una clase es una representación de una idea o concepto. Unidad que encapsula códigos y datos para los métodos (operaciones).
Todos los ejemplares de una clase se comportan de forma similar (invocan el mismo método) en respuesta a mensajes similares.

La clase a la cual pertenece un objeto determina el comportamiento del objeto.
Una clase tiene encomendadas actividades que ejecutan los métodos.

Las clases están definidas por:

- Atributos (Propiedades),
- Comportamiento (operaciones o métodos) y
- Relaciones con otros objetos.

Una aplicación es un conjunto de objetos de determinadas clases.

IES MARIA ENRIQUEZ

El lenguaje JavaScript no es un lenguaje orientado a objetos completo, pero permite definir clases con sus atributos y responsabilidades. Luego nos permite definir objetos de estas clases.

Pero el otro pilar de la programación orientada a objetos, es decir la herencia, recién se incorporó en las últimas versiones (lo veremos más adelante)

Veremos la sintaxis para la declaración de una clase y la posterior definición de objetos de la misma.

Desarrollaremos una clase que represente un cliente de un banco. La clase Cliente tiene como atributos:

nombre
saldo

y las responsabilidades o métodos de la clase son:

Constructor (inicializamos los atributos del objeto)
depositar
extraer

Luego debemos implementar los siguientes métodos (normalmente el constructor se utiliza el caracter mayúscula):

```
function Cliente(nombre,saldo)
{
  this.nombre=nombre;
  this.saldo=saldo;
  this.depositar=depositar;
  this.extraer=extraer;
}

function depositar(dinero)
{
  this.saldo=this.saldo+dinero;
}

function extraer(dinero)
{
  this.saldo=this.saldo-dinero;
}
```

El nombre de la clase coincide con el nombre de la función principal que implementamos (también llamado constructor de la clase):

```
function Cliente(nombre,saldo)
{
  this.nombre=nombre;
  this.saldo=saldo;
  this.depositar=depositar;
  this.extraer=extraer;
}
```

A ésta función llegan como parámetro los valores con que queremos inicializar los atributos. Con la palabra clave 'this' diferenciamos los atributos de los parámetros (los atributos deben llevar la palabra clave this)

```
  this.nombre=nombre;
  this.saldo=saldo;
```

También en el constructor inicializamos la referencia a todos los métodos que contendrá la clase (esto es muy importante y necesario para entender porque las otras dos funciones pertenecen a esta clase):

```
  this.depositar=depositar;
  this.extraer=extraer;
```

Por último, implementamos todos los métodos de la clase:

```
function depositar(dinero){
  this.saldo=this.saldo+dinero;
}

function extraer(dinero){
  this.saldo=this.saldo-dinero;
}
```

De nuevo recordemos que diferenciamos los atributos de la clase por la palabra clave this.

Ahora veamos el archivo HTML completo donde además definiremos un objeto de la clase planteada:

```
<!DOCTYPE html>
<html>

<head>
    <title>Ejemplo de JavaScript</title>
    <meta charset="UTF-8">
</head>
```

IES MARIA ENRIQUEZ

```
<body>

    <script>
        function Cliente(nombre, saldo) {
            this.nombre = nombre;
            this.saldo = saldo;
            this.depositar = depositar;
            this.extraer = extraer;
        }

        function depositar(dinero) {
            this.saldo = this.saldo + dinero;
        }

        function extraer(dinero) {
            this.saldo = this.saldo - dinero;
        }

        let cliente1;
        cliente1 = new Cliente('diego', 1200);
        document.write('Nombre del cliente:' + cliente1.nombre + '<br>');
        document.write('Saldo actual:' + cliente1.saldo + '<br>');
        cliente1.depositar(120);
        document.write('Saldo luego de depositar $120---->' + cliente1.saldo +
'<br>');
        cliente1.extraer(1000);
        document.write('Saldo luego de extraer $1000---->' + cliente1.saldo +
'<br>');
    </script>

</body>

</html>
```

Para definir un objeto de la clase Cliente tenemos:

```
let cliente1;
cliente1 = new Cliente('diego', 1200);
```

Luego las llamadas a métodos le antecedemos el nombre del objeto llamado cliente1:

```
document.write('Nombre del cliente:' + cliente1.nombre + '<br>');
document.write('Saldo actual:' + cliente1.saldo + '<br>');
cliente1.depositar(120);
```

```
document.write('Saldo luego de depositar $120---->' + cliente1.saldo + '<br>');
cliente1.extraer(1000);
document.write('Saldo luego de extraer $1000---->' + cliente1.saldo + '<br>');
```

Podemos decir que la ventaja que podemos obtener con el planteo de clases es hacer nuestros programas mucho más organizados, entendibles y fundamentalmente poder reutilizar clases en distintos proyectos.

Podemos directamente dentro del constructor implementar los dos métodos con la siguiente sintaxis:

```
function Cliente(nombre, saldo) {
    this.nombre = nombre;
    this.saldo = saldo;
    this.depositar = function(dinero) {
        this.saldo = this.saldo + dinero;
    }
    this.extraer = function(dinero) {
        this.saldo = this.saldo - dinero;
    }
}
```

Luego no hay cambios cuando creamos un objeto de la clase Cliente.

IES MARIA ENRIQUEZ